

CHAPTER 7:

GRAPHICAL USER INTERFACE

Property of German Malaysian Institute

Contents

1. Introduction to Java AWT & Java Swing
2. Create the Menu Bar
3. Adding Jcomponent (Jlabel, JTextBox, JTextField, JTextArea, JComboBox, JRadioButton, JCheckBox)
4. Using Java Component properties (Layout Manager, Borders)
5. Jcomponent Event and Interactivity

Property of German Malaysian Institute

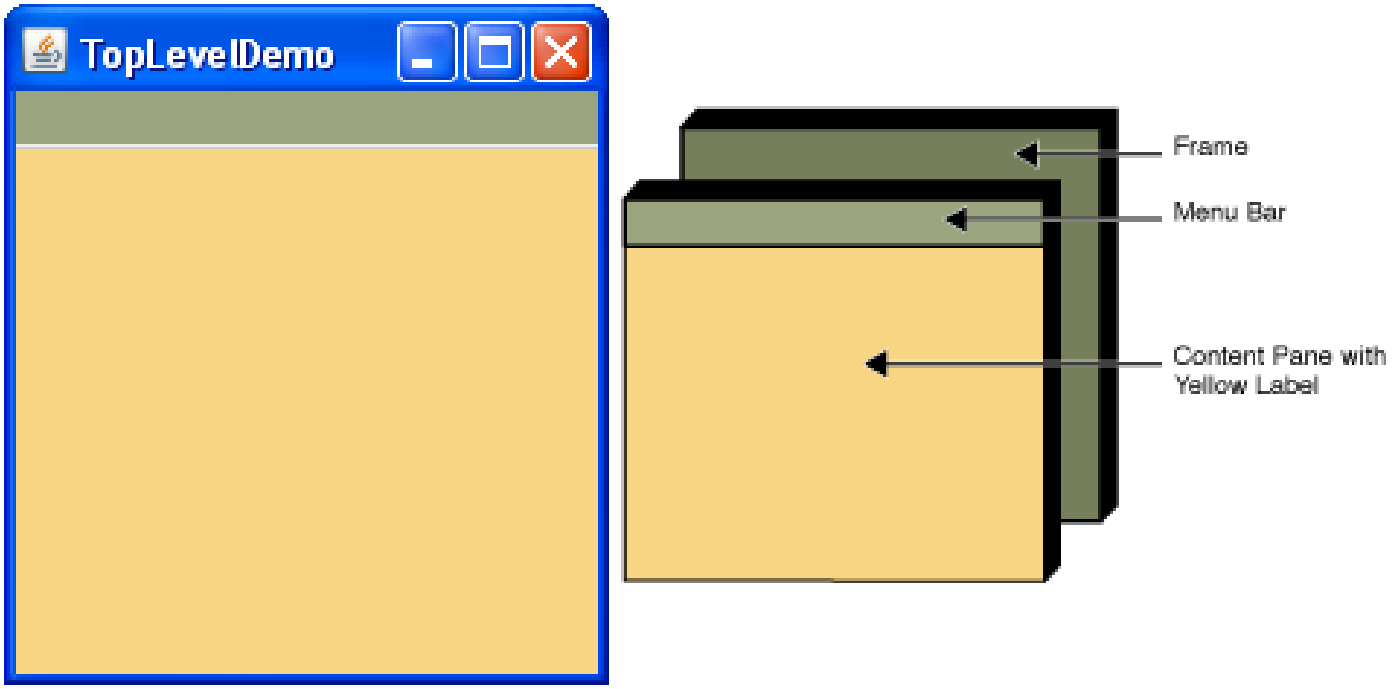
Introduction to GUI

- To appear onscreen, every GUI component must be part of a top-level container
- Each GUI component can be contained only once
- Each top-level container has a content pane that, generally speaking, contains (directly or indirectly) the visible components in that top-level container's GUI.
- Can optionally add a menu bar to a top-level container. The menu bar is by convention positioned within the top-level container.

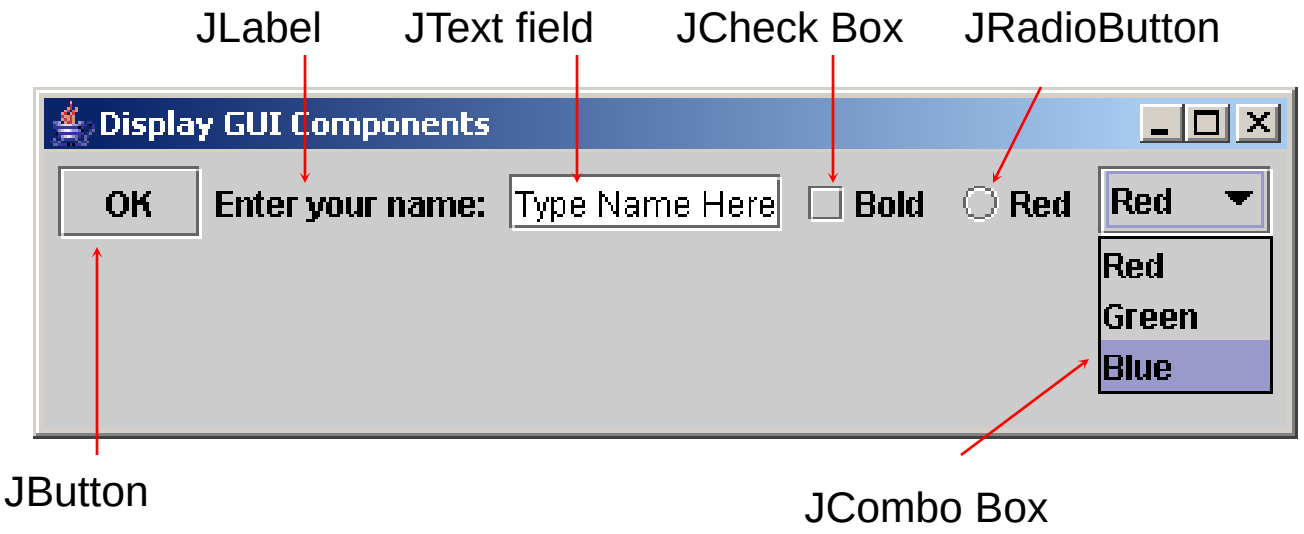
- **Top level container are**
  - **JFRAME**
  - **JAPPLET**
  - **JDIALOG**

Property of German Malaysian Institute

Creating Windows



JAVA Swing Components



- This subject will cover **JBUTTON, JLABEL,JTEXTFIELD, JTEXTAREA, JCHECKBOX, JCOMBOBOX, JRADIOBUTTON**

- It will also cover the Swing properties, such **as dimension, color, position, border style and Layout manager**

Property of German Malaysian Institute

JAVA Swing & AWT

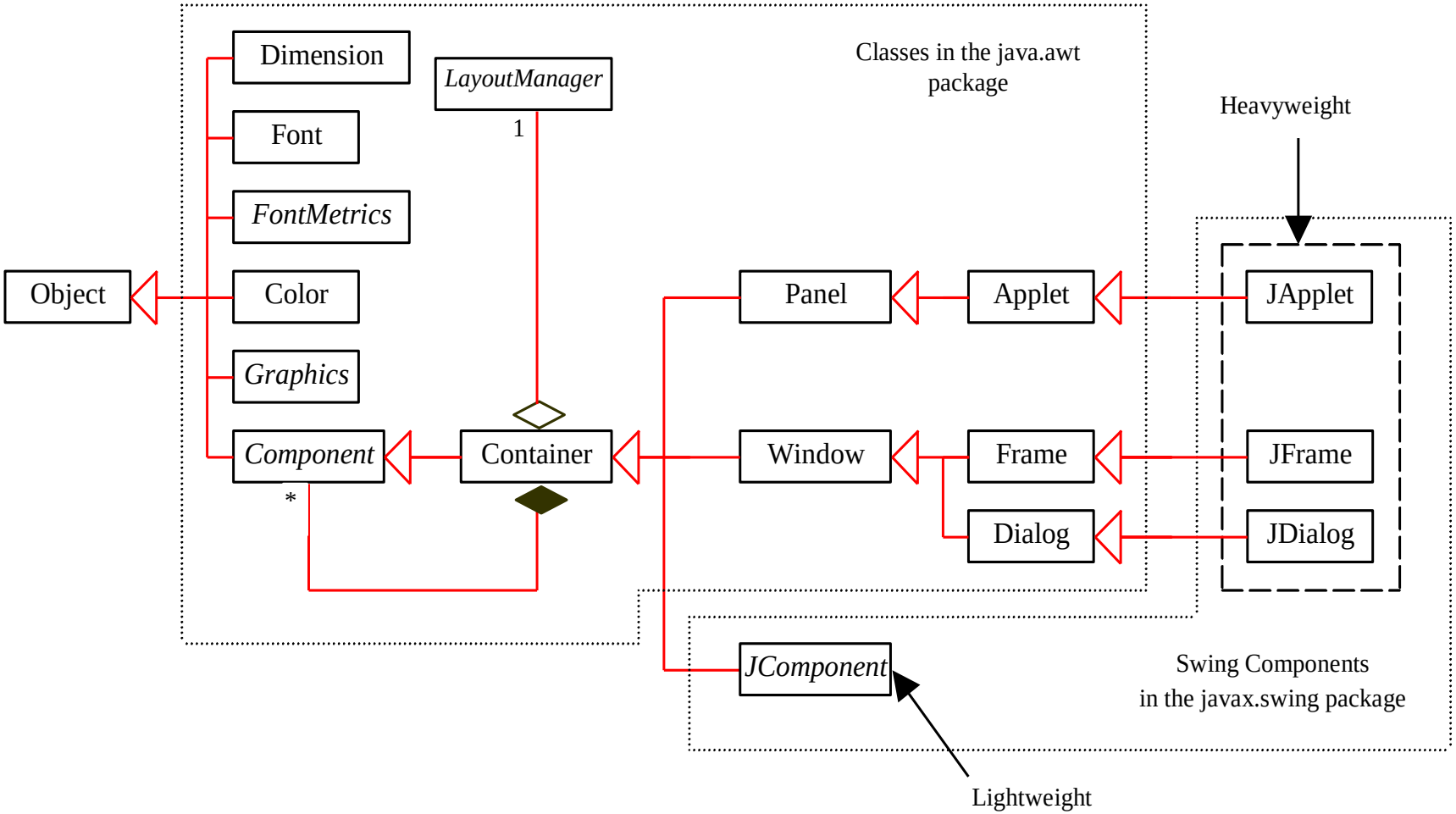
Why do the GUI component classes have a prefix *J*? Instead of JButton, why not name it simply Button? In fact, there is a class already named Button in the java.awt package.

When Java was introduced, the GUI classes were bundled in a library known as the Abstract Windows Toolkit (AWT). AWT is fine for **developing simple graphical user interfaces**, but not for developing comprehensive GUI projects. Besides, AWT is prone to platform-specific bugs because its peer-based approach relies heavily on the underlying platform.

With the release of Java 2, the AWT user-interface components were replaced by a **more robust, versatile, and flexible library** known as *Swing components*. Swing components are less dependent on the target platform and use less of the native GUI resource.

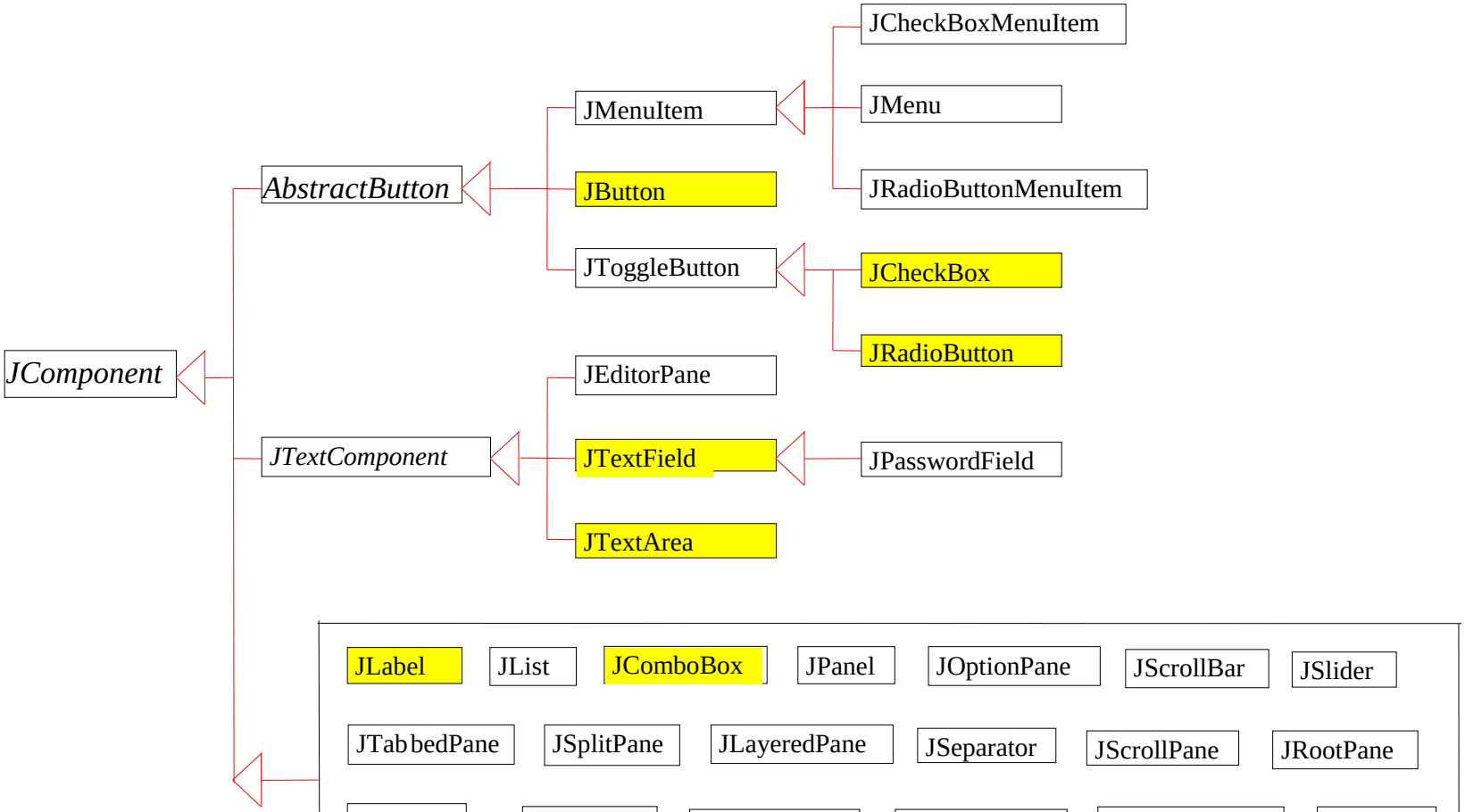
Property of German Malaysian Institute

JAVA Swing



Property of German Malaysian Institute

JAVA Swing GUI Component



Property of German Malaysian Institute

JButton

```
private JButton button1, button2;

Icon drink1 = new ImageIcon("milo.gif");
button1 = new JButton("Milo", drink1);
button2 = new JButton("Nescafe");
button1.setSize(100,40); button2.setSize(100,40);
button1.setLocation(20,30); button2.setLocation(20,80);
add(button1); add(button2);
```

JLabel

```
private JLabel Label1;

Label1 = new JLabel("Label with text");
Label1.setSize(200,30);
Label1.setLocation(20,30);
Label1.setForeground(Color.RED);
add(Label1);
```

Property of German Malaysian Institute

JTextField

```
private JTextField TextA;

TextA = new JTextField ("Key in here");
TextA.setSize(100,20);
TextA.setLocation(20,20);
TextA.setBackground(Color.red);
TextA.setForeground(Color.BLUE);
TextA.setFont(new Font("Courier",Font.ITALIC,15));
add(TextA);
```

JTextArea

```
private JTextArea Textbox1;

Textbox1 = new JTextArea ("Multiple line");
Textbox1.setSize(100,200);
Textbox1.setLocation(20,20);
Textbox1.setBackground(Color.PINK);
Textbox1.setForeground(Color.DARK_GRAY);
Textbox1.setFont(new
Font("Bookman",Font.TRUETYPE_FONT,16));
add(Textbox1);
```

Property of German Malaysian Institute

JRadioButton

```
private JRadioButton b1, b2, b3;

ButtonGroup group1 = new ButtonGroup();

b1 = new JRadioButton("Hagen Dazz");
b2 = new JRadioButton("Basken Robin");
b3 = new JRadioButton("Walls");

b1.setBounds(20,20,200,50);
b2.setBounds(20,70,200,50);
b3.setBounds(20,110,200,50);

group1.add(b1);
group1.add(b2);
group1.add(b3);
```

Property of German Malaysian Institute

JComboBox

```
private JComboBox Combobox;

Private String names[]= {"kuih.gif","kuih1.gif","kuih3.gif"};
ComboBox = new JComboBox(names);
ComboBox.setMaximumRowCount(3);
ComboBox.setBounds(50,50,100,20);
add(ComboBox);
```

JCheckBox

```
private JCheckBox cb1, cb3 ;

cb1 = new JCheckBox("C");
cb2 = new JCheckBox("JAVA");

c1.setBounds(20,20,200,50);
c2.setBounds(20,70,200,50);

add(c1);
add(c2);
```

Property of German Malaysian Institute

Adding Menu Bar

```
JMenuBar Menubar1 ;
JMenu Menusub1,Menusub2,sub1;
JMenuItem itemlist1,itemlist2;

Menubar1 = new JMenuBar();

Menusub1=new JMenu ("File");
Menusub2=new JMenu ("Help");
sub1 = new JMenu();

itemlist1=new JMenuItem ("Open File");
Menusub1.add(itemlist1);
itemlist1=new JMenuItem ("Close File");
Menusub1.add(itemlist1);
itemlist1=new JMenuItem ("Save File");
Menusub1.add(itemlist1);

itemlist1=new JMenuItem ("About");
Menusub2.add(itemlist1);

sub1 =new JMenu ("Expand");
```

```
 //******Adding submenu*********//
itemlist2=new JMenuItem ("Expand 1");
sub1.add(itemlist2);
itemlist2=new JMenuItem ("Expand 2");
sub1.add(itemlist2);

 //******Finish adding submenu*********//
 Menusub2.add(sub1);
 itemlist1=new JMenuItem ("Exit");
 Menusub2.add(itemlist1);

 Menubar1.add(Menusub1);
 Menubar1.add(Menusub2);
```

Property of German Malaysian Institute

Layout Manager

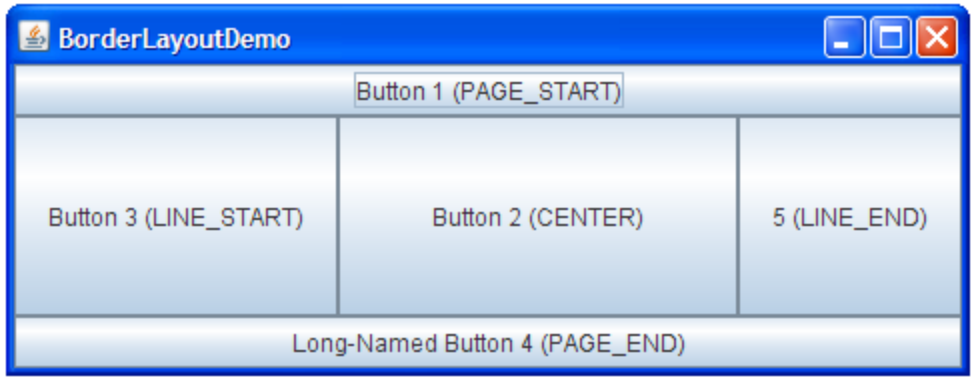
Java's layout managers provide a level of abstraction to automatically map interface on all window systems.

The UI components are placed in containers. Each container has a layout manager to arrange the GUI components within the container.

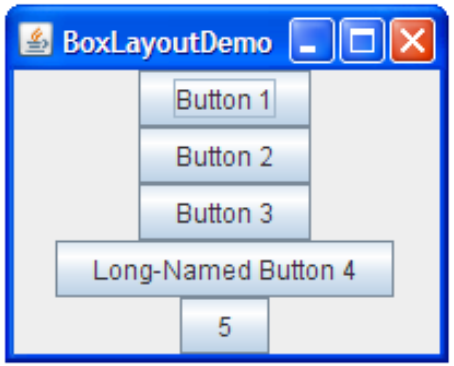
- BorderLayout
- BoxLayout
- CardLayout
- FlowLayout
- GridBagLayout
- GridLayout
- GroupLayout
- SpringLayout

Property of German Malaysian Institute

BorderLayout



BoxLayout

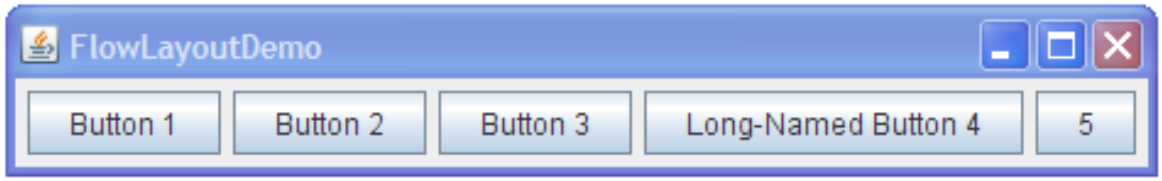


Property of German Malaysian Institute

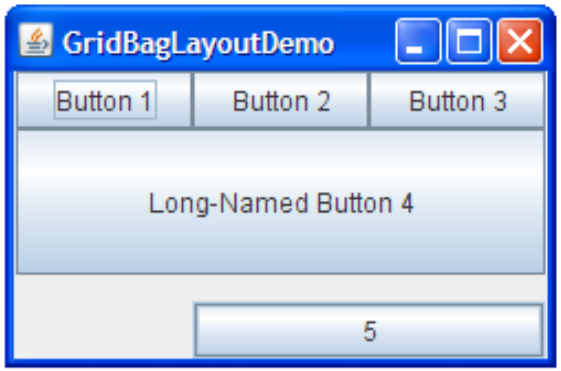
CardLayout



FlowLayout

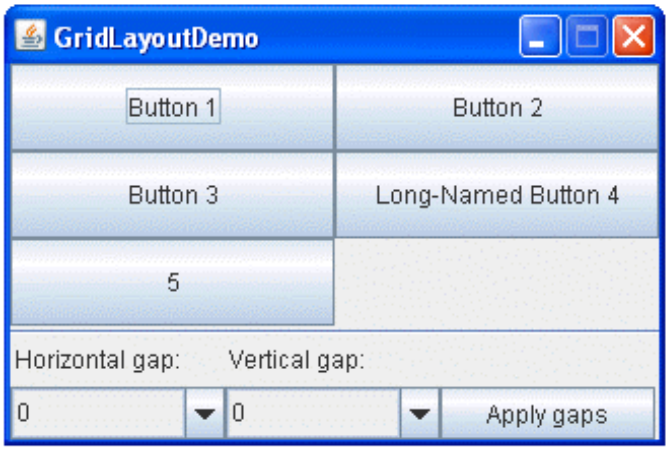


GridBagLayout

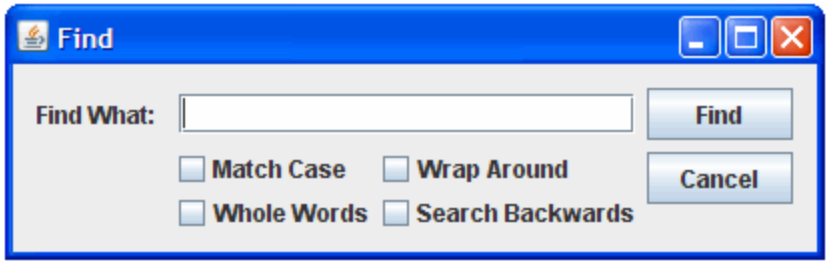


Property of German Malaysian Institute

GridLayout

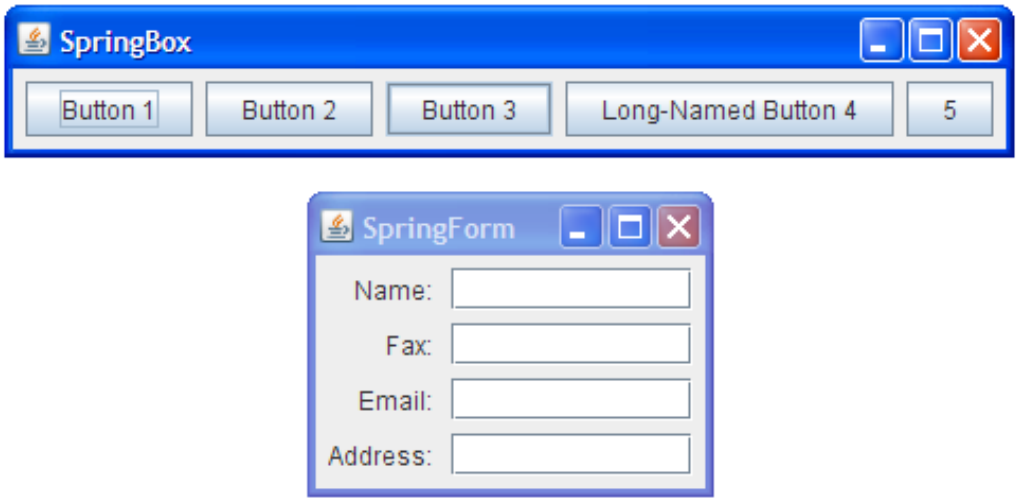


GroupLayout



Property of German Malaysian Institute

SpringLayout



Property of German Malaysian Institute

Borders

```
private JLabel Label1,Label2,Label3,Label4;

Label1 = new JLabel("This is border style 1");
Label1.setBorder(BorderFactory.createLineBorder(Color.BLUE));
add(Label1);

Label2 = new JLabel("This is border style 2");
Label2.setBorder(BorderFactory.createRaisedBevelBorder());
add(Label2);

Label3 = new JLabel("This is border style 3");
Label3.setBorder(BorderFactory.createEtchedBorder(1));
add(Label3);


Label4 = new JLabel("This is border style 4");
Label4.setBorder(BorderFactory.createTitledBorder("This is Title Border"));
add(Label4);
```

Property of German Malaysian Institute

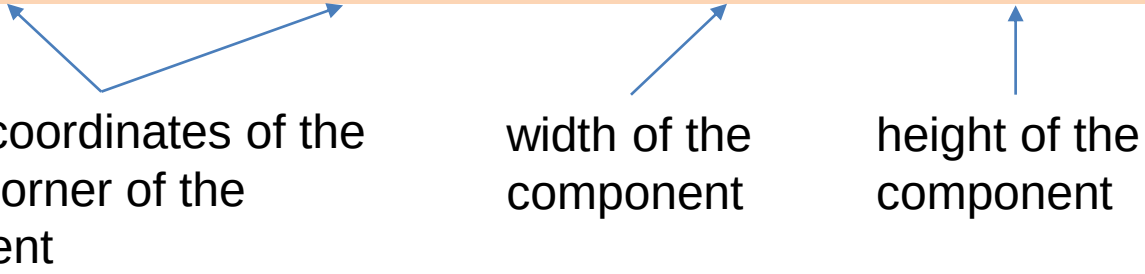
setBounds( )

The layout managers are used to automatically decide the position and size of the added components. In the absence of a layout manager, the position and size of the components have to be set manually.

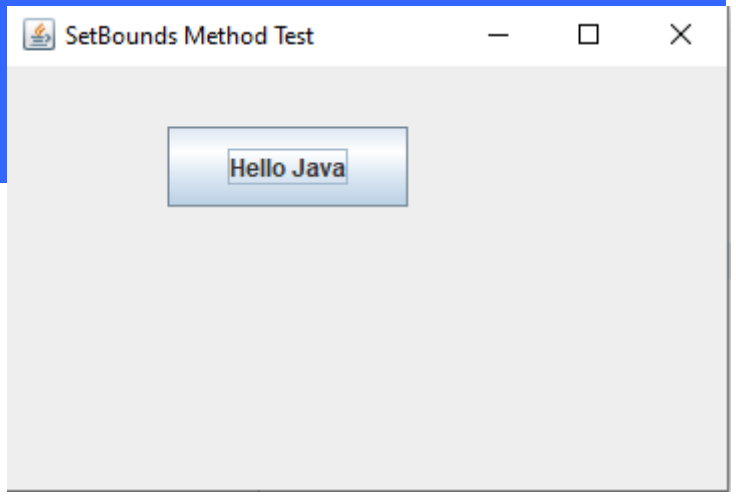
The setBounds() method is used in such a situation to set the position and size. To specify the position and size of the components manually, the layout manager of the frame can be null.

Syntax:

setBounds (int x-coordinate, int y-coordinate, int width, int height)



setBounds( )




```
import javax.swing.*;
import java.awt.*;

public class SetBound {

public static void main(String[] args) {
      JFrame frame = new JFrame("SetBounds Method Test");
      frame.setSize(375, 250);
      // Setting layout as null
      frame.setLayout(null);
      // Creating Button
      JButton button = new JButton("Hello Java");
      // Setting position and size of a button
      button.setBounds(80,30,120,40);
      frame.add(button);
      frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
     // frame.setLocationRelativeTo(null);
      frame.setVisible(true);
    }
}
```

Tabbed Panes

Create

```
jtp = new JTabbedPane();
jtp.addTab(“Cities”, new CitiesPanel());
jtp.addTab(“Colors”, new ColorsPanel());
add(jtp);
```

Create new class for every panel

```
class CitiesPanel extends Jpanel {
   public CitiesPanel(){
        .
        . }
}
```

Event & Interactivity

An event can be defined as a type of signal to the program that something has happened.

The event is generated by external user actions such as **mouse movements**, **mouse clicks**, and **keystrokes**, or by the operating system, such as a **timer**.

Property of German Malaysian Institute

JButton Event & Interactivity

```
public class Arithmetic2 extends JFrame implements
ActionListener  {
       private JButton button1;
       button1 = new JButton("Addition");
       button1.addActionListener(this);
}

public void actionPerformed(ActionEvent event)
{
   String buttonString = event.getActionCommand();

   if (buttonString.equals("Addition"))

       field.settext("Congratulation!!!");

}
```

Property of German Malaysian Institute

JCheckBox Event & Interactivity

```
public class UsingJCheckbox extends JPanel implements ItemListener

private JCheckBox chk1,chk2;

chk1 = new JCheckBox("Show Label 1");
chk1.addItemListener(this);



public void itemStateChanged(ItemEvent e)
{
      Object detect = e.getItemSelectable();
      if (detect==chk1)
      {
         if (e.getStateChange()==ItemEvent.SELECTED)
         {

         }
       }
}
```

JRadioButton Event & Interactivity

```
public class UsingJRadio extends JPanel implements ActionListener {

private JRadioButton radio1,radio2;

radio1 = new JRadioButton("Label 1");
radio2 =new JRadioButton("Label 2");

radio1.setActionCommand("r1");
radio2.setActionCommand("r2");

radio1.addActionListener(this);
radio2.addActionListener(this);

ButtonGroup mygroup = new ButtonGroup();
mygroup.add(radio1);
mygroup.add(radio2);

}

public void actionPerformed(ActionEvent e)
   {
         if ("r1".equals(e.getActionCommand()))
         {
         }

        if ("r2".equals(e.getActionCommand()))
        {
        }
}
```

JComboBox Event & Interactivity

```
public class Combo1 extends JFrame implements ItemListener{

    private JComboBox ComboBox;
    private JLabel label;
    private String names[]= {"kuih.gif", "kuih1.gif","kuih3.gif"};
    private     Icon     icons[]=     {new     ImageIcon(names[0]),new
ImageIcon(names[1]),new ImageIcon(names[2])};

    public Combo1 (){
        super("Testing ComboBox");
        Container container = getContentPane();
        container.setLayout(new FlowLayout());

        ComboBox = new JComboBox(names);
        ComboBox.setMaximumRowCount(3);
        ComboBox.addItemListener(this);

        container.add(ComboBox);

        label = new JLabel(icons[0]);
        container.add(label);

        setSize(400,300);
        setVisible(true);    }

    public void itemStateChanged(ItemEvent event){
        if (event.getStateChange()== ItemEvent.SELECTED)
            label.setIcon(icons[ComboBox.getSelectedIndex()]);
 }
```

Property of German Malaysian Institute

Summary 1

User Action	Source Object	Event Type Generated
Click a button	JButton	ActionEvent
Click a check box	JCheckBox	ItemEvent, ActionEvent
Click a radio button	JRadioButton	ItemEvent, ActionEvent
Press return on a text field	JTextField	ActionEvent
Select a new item	JComboBox	ItemEvent, ActionEvent
Window opened, closed, etc.	Window	WindowEvent
Mouse pressed, released, etc.	Component	MouseEvent
Key released, pressed, etc.	Component	KeyEvent

Property of German Malaysian Institute

Summary 2

JButton

- implements **ActionListener**
- **setActionCommand**("r1");
- **addActionListener(this);**
- public void **actionPerformed(ActionEvent e)**
- **e.getActionCommand()**

JCheckBox

implements I**temListener**
addItemListener(this);
public void **itemStateChanged**(I**temEvent e**)
e.getItemSelectable();
e.getStateChange()==I**temEvent.SELECTED**)

Property of German Malaysian Institute

Summary 2

JRadioButton

- implements **ActionListener**
- **setActionCommand**("r1");
- **addActionListener(this);**
- public void **actionPerformed(ActionEvent e)**
- **e.getActionCommand()**

Property of German Malaysian Institute

Reading input in text field

```
no1=Integer.parseInt(field.getText());
```

Property of German Malaysian Institute